Robotic Navigation and Exploration
HW2: Kinematic Models and Path Tracking Control
Due Sunday, March 29, 2026 (11:59 PM)

Student ID: R76141154 **Name:** 江冠緯

To complete this assignment, make sure you have the following tools installed on your computer:

1. Python version 3.8 +
2. opencv-python
3. numpy
4. matplotlib

Your written response must be typeset (LaTeX) and in blue. Also, do not forget to convert it into `report.pdf` before submission.

Please organize your submission into a folder as follows:

```
[your student ID]_HW2/
└ report.pdf
└ code/
    └ ...
```

Then, compress this folder into a zip archive named `[your student ID]_HW2.zip` and submit it to NTU COOL.
Warning: A 10-point penalty will be applied if your submission does not follow this format.

1 Introduction

This assignment focuses on the implementation and simulation of three fundamental **wheeled vehicle kinematic models**, alongside various **path-tracking control algorithms**. The primary objective is to enable an autonomous robot to robustly track a reference trajectory (generated in HW1) within a simulated environment.

The control architecture is decoupled into two parallel systems: **Longitudinal Control** and **Lateral Control**. First, you will build kinematic models for Basic, Differential Drive, and Bicycle configurations. Subsequently, longitudinal control algorithms will be implemented to generate safe velocity profiles and regulate the vehicle's speed along the track. Finally, you will develop multiple lateral tracking controllers—including PID, Pure Pursuit, Stanley, and LQR—to precisely minimize cross-track and heading errors. The assignment concludes by deploying these algorithms on simulated F1 circuits to evaluate their performance.

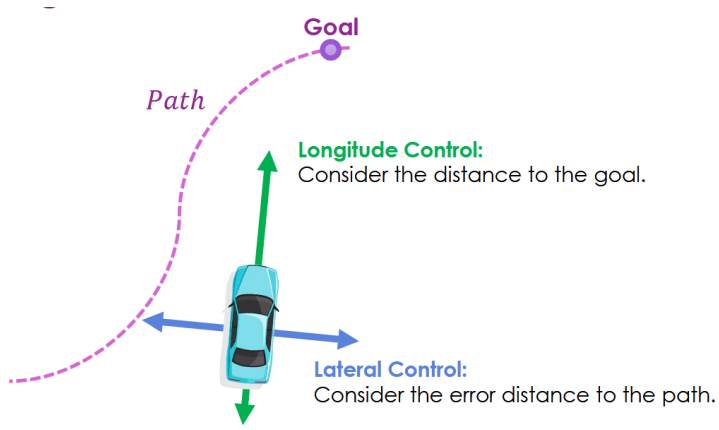


Figure 1: Longitudinal Control and Lateral Control

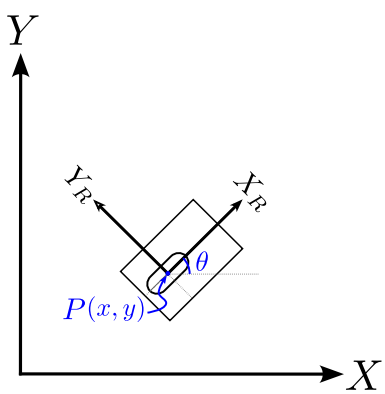
2 Kinematic Models

To simulate realistic vehicle motion, this section formulates the state-update logic for three distinct kinematic models. Each mathematical model dictates how the vehicle's **state** $\boldsymbol{\zeta} = [x, y, \theta]^T$ evolves continuously subject to a given **control input** $\boldsymbol{u}$.

In the provided codebase, the `State` object encapsulates not only the pose (x, y, θ) but also the vehicle's speed v and angular velocity ω. This formulation is particularly advantageous for models such as the bicycle model, where the direct control inputs are longitudinal acceleration a and steering angle δ; maintaining these dynamic states simplifies the discrete-time kinematic update. Note that throughout this document, (x, y) denotes the 2D position, and θ represents the global heading (orientation).

2.1 Basic Model

The basic model assumes that the robot can be approximated as a single wheel rolling on a plane. It can move forward or rotate but cannot move sideways.



- Control State $\boldsymbol{u} = [v, \omega]^T$, where v is the linear velocity and ω is the angular velocity.
- State: $\boldsymbol{\zeta} = [x, y, \theta]^T$, where x, y is the position and θ is the orientation of the vehicle.

$$\dot{\boldsymbol{\zeta}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = R(\theta) \begin{bmatrix} \dot{x_R} \\ \dot{y_R} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ 0 \\ \omega \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix} \tag{1}$$

1. Check the code in `Simulation/kinematic_basic.py`, and answer the following questions.
   (a) (2 pts) What is the unit of `yaw`?
   Yaw is in degrees, since the update uses trig and the wrap is % 360
   (b) (2 pts) Why do we need `yaw = (state.yaw + state.w * self.dt) % 360`? What would happen without this line?

In this line of code, it keeps yaw normalized to [0,360) so it doesn' t grow without bound or go negative; without it, yaw can drift outside the expected range and break any logic/visualization that assumes normalized degrees

2. (2 pts) In practice, a controller may output infeasible commands (e.g., exceeding the actuator limits). Therefore, our simulator applies constraints to the current state and/or control inputs. Please check the code in `Simulation/simulator_basic.py` and list the constraints that are enforced.

   - **Linear acceleration constraint (rate of change of v):**
     $\Delta v = v_k - v_{k-1}$ is limited each step by

     $$\Delta v \in [a_{\min}\, dt,\ a_{\max}\, dt]\,,$$

     where $a_{\min} = -20$, $a_{\max} = 15$, and $dt = 0.05$.
   - **Linear velocity constraint:**

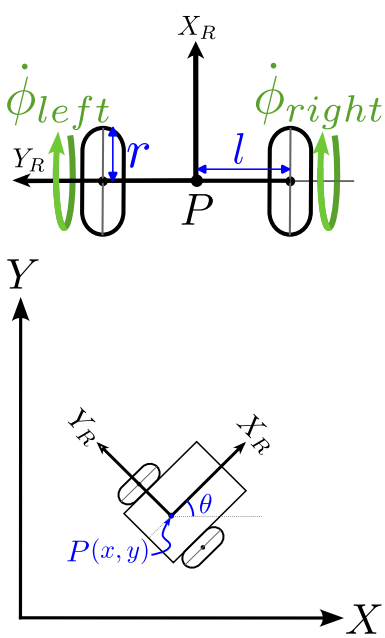
     $$v \in [-v_{\max},\ v_{\max}]\,, \quad v_{\max} = 90.$$

   - **Angular velocity constraint:**

     $$\omega \in [-\omega_{\max},\ \omega_{\max}]\,, \quad \omega_{\max} = 180.$$

2.2 Differential Drive Model

The differential drive model features two independently driven wheels on a shared axis. In this model, the control input becomes $[\dot{\phi}_{\text{left}}, \dot{\phi}_{\text{right}}]^T$, representing the angular velocities of the left and right wheels, respectively.



- Control Input $\boldsymbol{u} = [\dot{\phi}_{\text{left}}, \dot{\phi}_{\text{right}}]^T$ The control input can be mapped to $[v, \omega]$ by the following formula:

  $$v = \frac{r}{2}(\dot{\phi}_{\text{left}} + \dot{\phi}_{\text{right}})$$
  $$\omega = \frac{r}{2l}(\dot{\phi}_{\text{right}} - \dot{\phi}_{\text{left}})$$

- State: $\boldsymbol{\zeta} = [x, y, \theta]^T$, where x, y is the position and θ is the orientation of the vehicle.

  $$\dot{\boldsymbol{\zeta}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix} = \begin{bmatrix} \frac{r}{2}(\dot{\phi}_{\text{left}} + \dot{\phi}_{\text{right}})\cos\theta \\ \frac{r}{2}(\dot{\phi}_{\text{left}} + \dot{\phi}_{\text{right}})\sin\theta \\ \frac{r}{2l}(\dot{\phi}_{\text{right}} - \dot{\phi}_{\text{left}}) \end{bmatrix} \tag{2}$$

Related code in `Simulation/kinematic_different_drive.py`

For more details and derivations, please refer to the lecture slides (pp.24).

1. (2 pts) Similar to 2.1.2, check the code in `Simulation/simulator_different_drive.py` and list the constraints that are enforced.

- **Wheel angular acceleration (rate-of-change) constraints:**

$$lw_k - lw_{k-1} \in \left[-\dot{lw}_{\max}\, dt,\ \dot{lw}_{\max}\, dt\right], \quad rw_k - rw_{k-1} \in \left[-\dot{rw}_{\max}\, dt,\ \dot{rw}_{\max}\, dt\right],$$

  where $\dot{lw}_{\max} = \texttt{dot_lw_range} = 8000$, $\dot{rw}_{\max} = \texttt{dot_rw_range} = 8000$, and $dt = 0.05$.

- **Wheel angular velocity constraints:**

$$lw_k \in \left[-lw_{\max},\ lw_{\max}\right], \quad rw_k \in \left[-rw_{\max},\ rw_{\max}\right],$$

  where $lw_{\max} = \texttt{lw_range} = 34400$ and $rw_{\max} = \texttt{rw_range} = 34400$.

2. (4 pts) Complete `## TODO 2.2.2` in `navigation.py`. For the following lateral control algorithm, the basic and differential drive models are implemented in the same file. However, for the differential drive model, you need to convert the control input $[v, \omega]$ into $[\dot{\phi}_{\text{left}}, \dot{\phi}_{\text{right}}]$. Run the command below to check the result of differential drive model:

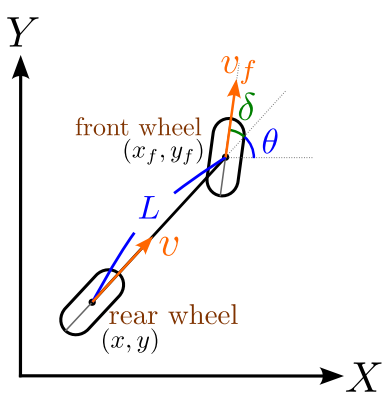
```
$ python navigation.py -s diff_drive -c pid -t 400mRunningTrack
```

   you should have similar result by running the following command:

```
$ python navigation.py -s basic -c pid -t 400mRunningTrack
```

2.3 Bicycle Model

The bicycle model is a simplified mathematical representation of a four-wheeled vehicle's dynamics, reducing it to two wheels (one front, one rear) on a single track.



- Control Input $\boldsymbol{u} = [a, \delta]^T$, where a is the linear acceleration and δ is the steering angle.
- State: $\boldsymbol{\zeta} = [x, y, \theta]^T$, where x, y is the position and θ is the orientation of the vehicle.

$$\dot{\boldsymbol{\zeta}} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \omega \end{bmatrix} = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ \frac{v}{L}\tan\delta \end{bmatrix} \tag{3}$$

  where v is the velocity, L is the wheelbase, and δ is the steering angle.

For more details and derivations, please refer to the lecture slides (pp.32).

1. (5 pts) According to the above formula, complete the `step()` function in the `KinematicModelBicycle` class which is defined in the file `Simulation/kinematic_bicycle.py`

```
$ python navigation.py -s bicycle -c pid
```

2. (3 pts) Similar to 2.1.2, check the code in `Simulation/simulator_bicycle.py` and list the constraints that are enforced.

   - **Acceleration constraint:**

$$a \in \left[a_{\min},\ a_{\max}\right], \quad a_{\min} = -20,\ a_{\max} = 15.$$

- **Steering rate constraint (per step):**

$$\delta_k - \delta_{k-1} \in \left[-\dot{\delta}_{\max},\, \dot{\delta}_{\max}\right], \quad \dot{\delta}_{\max} = 25.$$

- **Steering angle constraint:**

$$\delta \in [-\delta_{\max},\, \delta_{\max}], \quad \delta_{\max} = 40.$$

- **Speed constraint (state):**

$$v \in [-v_{\max},\, v_{\max}], \quad v_{\max} = 90.$$

3 Path Tracking - Longitudinal Control

In Homework 1, you generated the path geometry (waypoints). However, a raw path isn't enough for tracking; the vehicle needs a velocity profile to know when to slow down for corners and stay stable.

1. Following the methodology below, complete the `generate_speed_profile()` function in `trajectory_generator.py`

   Using these (x, y) waypoints, compute a reference velocity profile v_{ref} for the entire trajectory. The velocity profile should be both **safe** (limited by curvature) and **dynamically feasible** (smooth acceleration/deceleration).

   Algorithm outline:

   (a) **Curvature κ:** Compute the curvature at each waypoint from the derivatives of the path coordinates. (already written)

   (b) **Speed limit (4 pts):** Compute the maximum allowable speed based on lateral acceleration ($a_{\text{lat}} = 30\text{m/s}^2$) and a global speed cap ($v_{\max} = 85\text{m/s}$):

   $$v_{\text{limit}} = \min\left(v_{\max}, \sqrt{\frac{a_{\text{lat}}}{|\kappa|}}\right).$$

   Hint: In practice, κ can be zero on straight segments. To avoid division by zero, add a small constant (e.g., $\epsilon = 10^{-6}$) to the denominator: $|\kappa| + \epsilon$.

   (c) **Longitudinal smoothing (8 pts):** Apply a forward–backward pass ensuring the velocity profile respects acceleration and braking limits.

   - **Forward pass:** limit acceleration ($a_{\text{acc}} = 12\text{m/s}^2$).

   $$v_i^2 \le v_{i-1}^2 + 2 \cdot a_{\text{acc}} \cdot ds$$

   - **Backward pass:** limit deceleration ($a_{\text{dec}} = 18\text{m/s}^2$) so the vehicle slows down before corners.

   $$v_i^2 \le v_{i+1}^2 + 2 \cdot a_{\text{dec}} \cdot ds$$

   - **Launch Restrictions:**
     - Starting Speed ($v_{\text{ref}}[0]$): 5.0 m/s
     - Terminal Target Speed ($v_{\text{ref}}[-1]$): 0.0 m/s

Hint: Think carefully about where to apply each restriction.

Run the following command to check your velocity profile:

```
$ python trajectory_generator.py
```

2. (8 pts) **Speed Tracking with PID:** In our bicycle model, the longitudinal control input is the vehicle's acceleration a. Please implement a PID controller that calculates the appropriate acceleration to effectively track the reference speed v_{ref}. The default gains K_p, K_i, and K_d are already provided in the code, but feel free to fine-tune them!

   PID formula:

$$e(t) = v_{\text{ref}}(t) - v(t), \qquad u(t) = a = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d\, \frac{de(t)}{dt}.$$

   Run the following command to check your longitudinal PID controller:

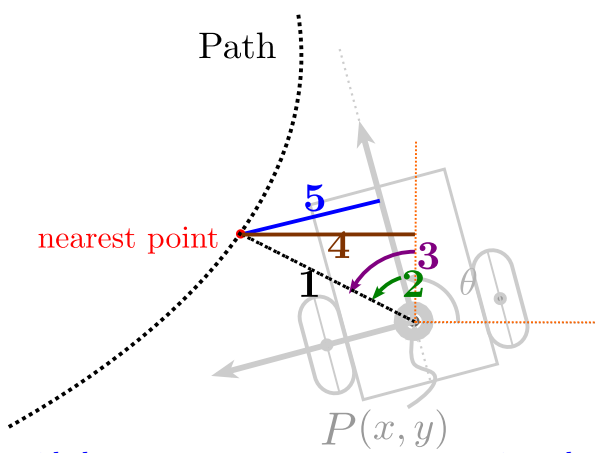
```
$ python navigation.py -s bicycle -c pid -t 1000mStraight
```

4 Path Tracking - Lateral Control

4.1 PID Control

PID control works just as well for lateral tracking. The main difference is simply how we define the tracking error.

1. (2 pts) According to `PathTracking/controller_pid_basic.py`, identify the error term of PID lateral control.



5. According to the provided `controller_pid_basic.py` script, the error term utilized for the PID lateral control is the cross-track error, mathematically defined in the code as err = min_dist $\cdot \sin(\theta_{\text{err}})$. Correlating this code with the provided geometric diagram confirms the following assessment: the line segment labeled as **1** represents the variable min_dist, which is the direct straight-line distance from the robot's center point $P(x, y)$ to the nearest point on the reference path, and the segment labeled as **5** represents the final evaluated error term min_dist $\cdot \sin(\theta_{\text{err}})$, which geometrically corresponds to the perpendicular lateral deviation of the vehicle relative to its target heading.

2. (5 pts) Tune the PID gains K_p, K_i, and K_d in `PathTracking/controller_pid_basic.py` to achieve the best performance, and briefly describe how you tuned the PID gains K_p, K_i, and K_d. It is recommended to include a screenshot of the resulting trajectory.
Hint: You may refer to PID Tuning Guide (or any other PID-tuning method).

```
$ python navigation.py -s basic -c pid -t 1000mStraight --init_shift 1.5
```

I tuned the gains in `controller_pid_basic.py` using a practical trial-and-error PID tuning process inspired by the TLK Energy PID tuning guide (PID Tuning Guide). In simulation, I first adjusted K_p to make the vehicle respond to the lateral error $e = \texttt{min_dist} * \sin(\theta_{\text{err}})$ quickly enough to converge toward the reference path, while avoiding overly aggressive oscillation; after obtaining a reasonable proportional response, I introduced a very small K_i to reduce the remaining steady-state offset observed during tracking; finally, I increased K_d to suppress oscillation and smooth the transient response when the vehicle corrected its lateral deviation. Based on these simulation results, I selected the final gains $K_p = 2.0$, $K_i = 0.02$, and $K_d = 0.4$, which provided stable and satisfactory path-tracking performance.

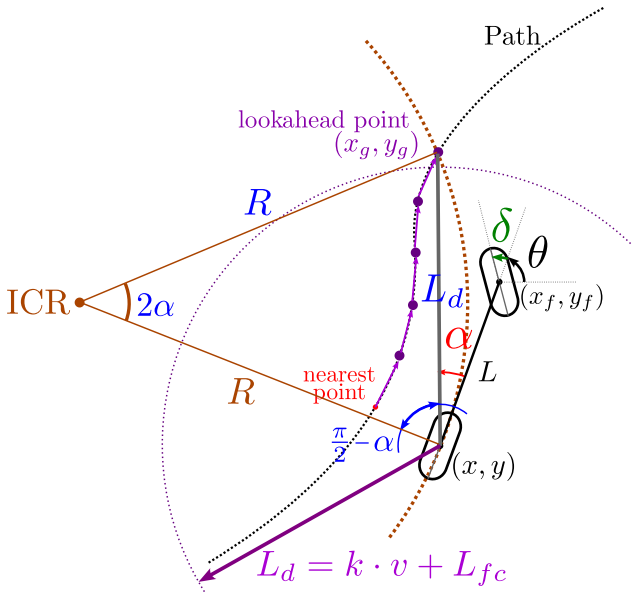
3. (5 pts) For the bicycle model, you may choose whether to use the same error definition as the basic model. If not, please describe the error you used. Complete `PathTracking/controller_pid_bicycle.py`
Run the following command to check your PID controller:

```
$ python navigation.py -s bicycle -c pid -t 400mRunningTrack
```

For the bicycle model, I chose the same lateral error definition as in the basic model, namely $e = d * \sin(\theta_{\text{err}})$, where d is the distance from the vehicle to the nearest point on the reference path and $\theta_{\text{err}} = \theta_{\text{target}} - \psi$ is the angle between the vehicle heading and the direction toward that nearest target point, because this error directly represents the signed cross-track deviation of the vehicle from the path and therefore provides a simple, intuitive, and consistent quantity for PID steering control; although the bicycle model has steering dynamics through the front-wheel angle δ, the control objective is still to reduce lateral deviation from the path, so reusing the same cross-track error definition keeps the controller design straightforward and makes the behavior easy to compare with the basic model.

4.2 Pure Pursuit Control

The Pure Pursuit algorithm calculates the steering angle δ needed to guide the vehicle toward a "look-ahead" point on the path.



Pure Pursuit Algorithm

1. Find the nearest point on the path.
2. Find the first point on the path that is at least L_d away.

$$L_d = k \times v + L_{fc}$$

3. Calculate the steering angle δ to move the vehicle toward the "look-ahead" point.

$$\delta = \arctan\left(\frac{2L\sin(\alpha)}{L_d}\right)$$

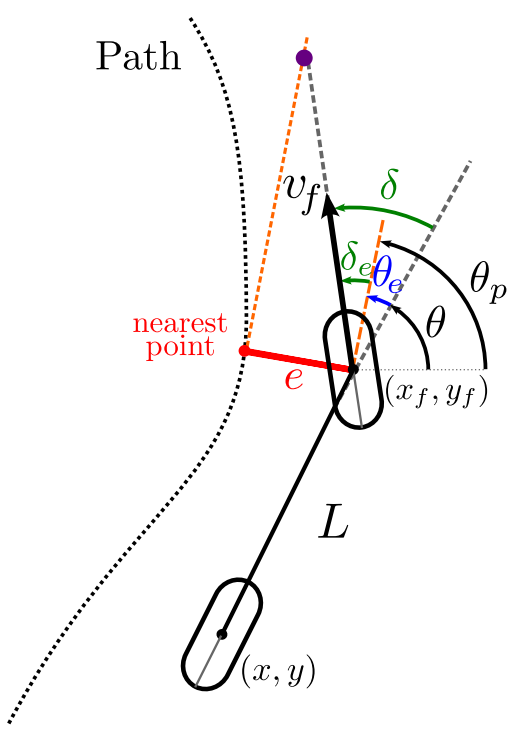
For more details and derivations, please refer to the lecture slides (pp. 34).

1. (8 pts) According to the above algorithm, complete `PathTracking/controller_pure_pursuit_bicycle.py`

```
$ python navigation.py -s bicycle -c pure_pursuit -t 400mRunningTrack
```

4.3 Stanley Control

Stanley control is a path-tracking algorithm that determines the steering angle by combining the **heading error** relative to the path with the **cross-track error** measured from the center of the front axle to the nearest point on the path.



Stanley Control Algorithm

1. Find the nearest point on the path.
2. Calculate the heading error θ_e and the cross-track error δ_e.

$$\theta_e = \theta_{path} - \theta$$

$$|e| = \sqrt{(x - x_{path})^2 + (y - y_{path})^2}$$

3. Calculate the steering angle δ to move the vehicle toward the "look-ahead" point.

$$\delta = \theta_e + \delta_e = \theta_e + \arctan\left(\frac{-ke}{v_f}\right) \tag{4}$$

For more details and derivations, please refer to the lecture slides (pp. 35).

1. (8 pts) According to the above algorithm, complete `PathTracking/controller_stanley_bicycle.py`

```
$ python navigation.py -s bicycle -c stanley -t 400mRunningTrack
```

Hint1: Pay attention to the magnitude of θ_e
Hint2: Be careful about the sign of e, where $\dot{e} = v_f \sin(\delta_e)$ in deviation of Eq. (4).

4.4 LQR Control

Problem Formulation

Given a discrete-time linear system:

$$\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k \tag{5}$$

The goal of LQR is to find an optimal control sequence $\mathbf{u}_k$ that minimizes the overall cost function J:

$$J = \sum_{k=0}^{\infty} (\mathbf{x}_k^T Q \mathbf{x}_k + \mathbf{u}_k^T R \mathbf{u}_k) \tag{6}$$

- Q (State Weight Matrix): Penalizes deviations from the desired state (error).
- R (Control Weight Matrix): Penalizes the control effort (energy expenditure).

The Matrix P (Cost-to-Go)

The matrix P is the key to solving LQR and represents the "Cost-to-Go".

- **Value Function $V(\mathbf{x})$:** The value function $V(\mathbf{x}_k)$ tells us the minimum cost from our current state $\mathbf{x}_k$ all the way to infinity:

$$V(\mathbf{x}_k) = \min_{\mathbf{u}_k, \mathbf{u}_{k+1}, \ldots} \sum_{i=k}^{\infty} \left(\mathbf{x}_i^T Q \mathbf{x}_i + \mathbf{u}_i^T R \mathbf{u}_i\right) \tag{7}$$

  For LQR, this value function is quadratic and is defined by the matrix P:

$$V(\mathbf{x}_k) = \mathbf{x}_k^T P \mathbf{x}_k \tag{8}$$

  Then, we can write the recursive form of value function:

$$\begin{aligned} V(\mathbf{x}_k) &= \min_{\mathbf{u}} \{\mathbf{x}_k^T Q \mathbf{x}_k + \mathbf{u}_k^T R \mathbf{u}_k + V(x_{k+1})\} \\ &= \min_{\mathbf{u}} \{\mathbf{x}_k^T Q \mathbf{x}_k + \mathbf{u}_k^T R \mathbf{u}_k + \mathbf{x}_{k+1}^T P \mathbf{x}_{k+1}\} \end{aligned} \tag{9}$$

- **Significance:**
  - P condenses the entire future trajectory dynamics and costs into a single matrix.
  - It measures the total future cost starting from state $\mathbf{x}_k$.

To find P, we must solve the Discrete Algebraic Riccati Equation (DARE):

$$P = A^T P A - (A^T P B)(R + B^T P B)^{-1}(B^T P A) + Q \tag{10}$$

The related code can be found in the function `_solve_DARE()` in `PathTracking/controller_lqr_bicycle.py`.

The Optimal Control Law

Once P is obtained, the optimal control input $\mathbf{u}_k$ is a linear state feedback law:

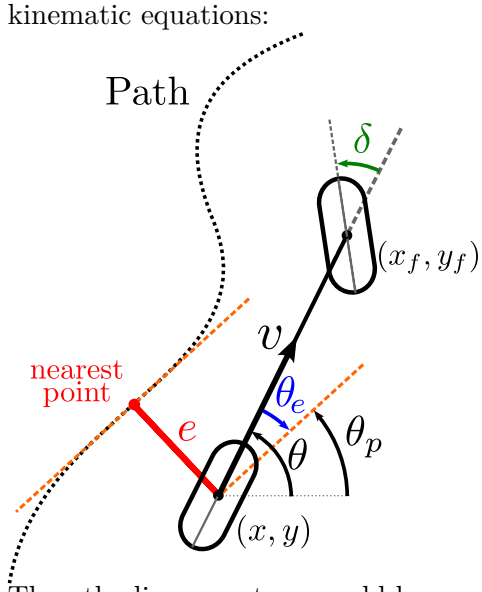
$$\mathbf{u}_k = -K\mathbf{x}_k \tag{11}$$

Where K is the optimal gain matrix derived from P:

$$K = (R + B^T PB)^{-1}(B^T PA) \tag{12}$$

The derivation process of K is detailed in lecture slides (pp.39).

1. (8 pts) We assume the state is $\mathbf{x} = [e, \theta_e]^T$, where e is the cross-track error and θ_e is the heading error, and the control input is $\mathbf{u} = [\delta]^T$. For the bicycle model, we have the following kinematic equations:



$$\dot{e} = -v\sin(\theta_e)$$
$$\dot{\theta}_e = -\frac{v}{L}\tan(\delta)$$

In LQR, we assume that θ_e and δ are small, so we can approximate $\sin(\theta_e) \approx \theta_e$ and $\tan(\delta) \approx \delta$, then we have

$$\dot{e} = -v\theta_e \tag{13}$$
$$\dot{\theta}_e = -\frac{v}{L}\delta \tag{14}$$

and

$$\begin{bmatrix} \dot{e} \\ \dot{\theta}_e \end{bmatrix} = \begin{bmatrix} 0 & -v \\ 0 & 0 \end{bmatrix} \begin{bmatrix} e \\ \theta_e \end{bmatrix} + \begin{bmatrix} 0 \\ -\frac{v}{L} \end{bmatrix} \delta$$

Then the linear system would be:

$$\begin{aligned} \mathbf{x}_{k+1} &= A\mathbf{x}_k + B\mathbf{u}_k \\ &= \begin{bmatrix} e_k \\ \theta_{e,k} \end{bmatrix} + \begin{bmatrix} \dot{e_k} \\ \dot{\theta}_{e,k} \end{bmatrix} \Delta t \\ &= \begin{bmatrix} 1 & -v \cdot \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} e_k \\ \theta_{e,k} \end{bmatrix} + \begin{bmatrix} 0 \\ -\frac{v}{L}\Delta t \end{bmatrix} \begin{bmatrix} \delta_k \end{bmatrix} \end{aligned}$$

Please refer to the equations above and complete the code in `PathTracking/controller_lqr_bicycle.py`

```
$ python navigation.py -s bicycle -c lqr -t 400mRunningTrack --lqr_control_state
    steering_angle
```

Hint1: Pay attention to the magnitude of θ_e
Hint2: Ensure that the definitions of θ_e, e, and δ in your code are consistent with Eq. (13) and (14).

2. (4 pts) Write down how you choose the weight matrices Q and R
I selected the LQR weight matrices by assigning a larger penalty to the cross-track error than

to the heading error in order to prioritize reducing lateral deviation from the path, while using a moderate control penalty to avoid overly aggressive steering commands; specifically, for the steering-angle formulation with state $x = [e, \theta_e]^T$ and control $u = [\delta]^T$, I chose $Q = \text{diag}(10,\ 3)$ and $R = \text{diag}(1)$, where the larger weight on e encourages the controller to quickly bring the vehicle back toward the path, the smaller but still nonzero weight on θ_e helps align the vehicle heading smoothly with the path direction, and the weight on δ limits excessive control effort so that the resulting steering behavior remains stable and not overly oscillatory.

3. (4 pts) Now, to regulate the steering speed, we need to change the control input $\mathbf{u}$ to the steering angular velocity, $\dot{\delta}$. The state then becomes $\mathbf{x} = [e, \theta_e, \delta]^T$.
Please finish the linear system below:

$$
\begin{aligned}
\mathbf{x}_{k+1} &= A\mathbf{x}_k + B\mathbf{u}_k \\
&= \begin{bmatrix} e_k \\ \theta_{e,k} \\ \delta_k \end{bmatrix} + \begin{bmatrix} \dot{e_k} \\ \dot{\theta}_{e,k} \\ \dot{\delta}_k \end{bmatrix} \Delta t \\
&= \begin{bmatrix} 1 & -v\Delta t & 0 \\ 0 & 1 & -\frac{v}{L}\Delta t \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} e_k \\ \theta_{e,k} \\ \delta_k \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \Delta t \end{bmatrix} \begin{bmatrix} \dot{\delta}_k \end{bmatrix}
\end{aligned}
$$

4. (6 pts) Based on the previous question, complete `PathTracking/controller_lqr_bicycle.py`

```
$ python navigation.py -s bicycle -c lqr -t 400mRunningTrack --lqr_control_state
    steering_angular_velocity
```

5 F1 Track Challenge

1. (10 pts) Now that you've built these models and controllers, it's time to put them to the test on some actual F1 circuits! We provide three F1 circuits: Silverstone, Suzuka, and Monza. You are free to choose whichever combination of kinematic model (basic, differential drive, or bicycle) and controller (PID, Pure Pursuit, Stanley, or LQR) that yields the best performance on average across all three tracks. You can tune controller parameters, implement additional controllers, or improve any part of the navigation flow to enhance overall system performance.

```
$ python navigation.py -s [basic|diff_drive|bicycle] -c [pid|pure_pursuit|stanley|
    lqr] -t [Silverstone|Suzuka|Monza]
```

Rules:

- Do not change the constraints $v_{\max}$, a_{lat}, a_{acc}, a_{dec} in velocity profiling or any simulator code.
- If you modify any code outside the designated `#TODO` blocks, please mark them with a `#NewFeature` comment and describe the changes in your report.
- Any violation of the above rules will receive 0 points in this challenge!

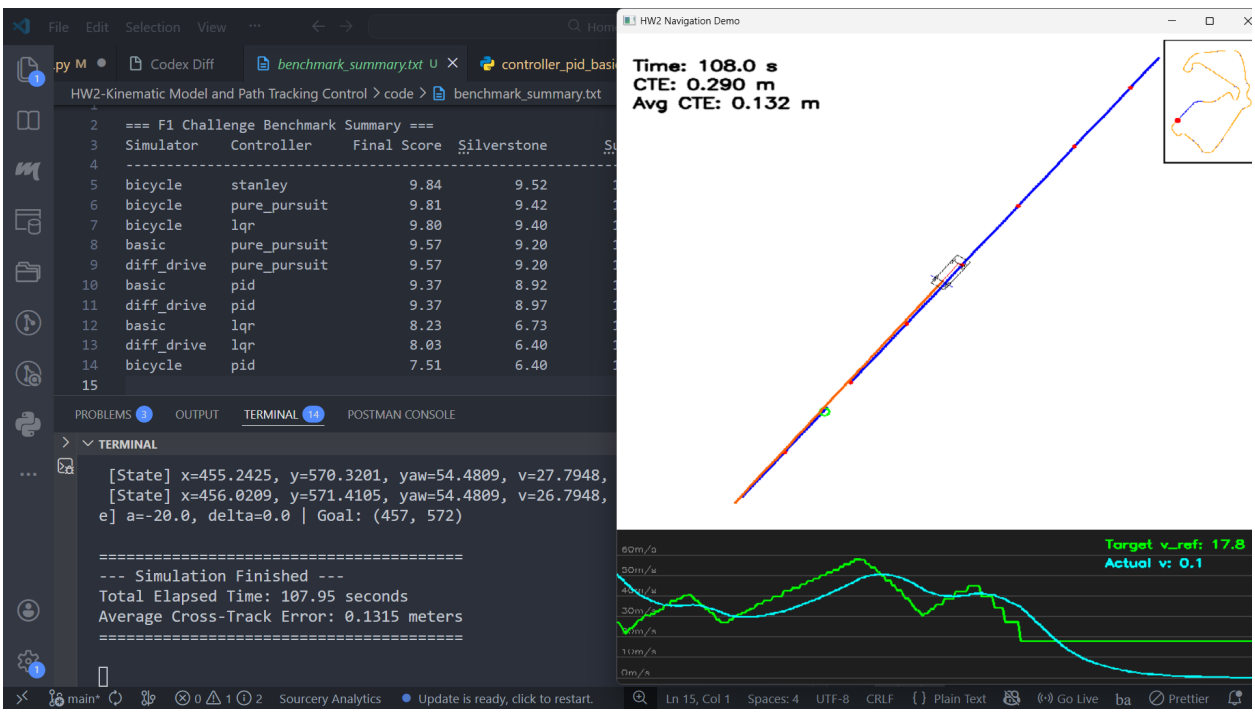
Grading Policy: Let $T_{\text{ref},i}$ be the baseline completion time for track i (**Silverstone: 107s, Suzuka: 115s, Monza: 94s**).

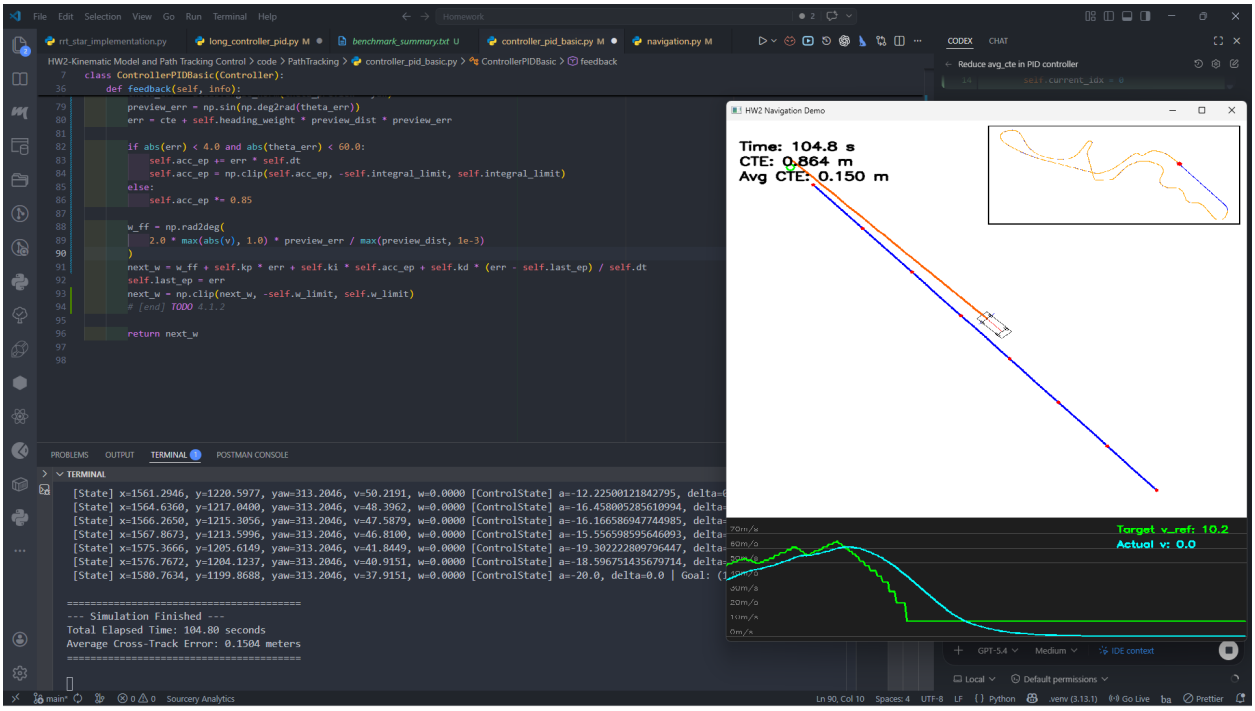
$$
\begin{aligned}
S_{T,i} &= \max\left(0,\ \min\left(6,\ 4 - k_T(T_i - T_{\text{ref},i})\right)\right) \\
S_{E,i} &= \max\left(0,\ \min\left(6,\ 4 - k_E(E_i - 1.5)\right)\right) \\
\text{Score}_i &= \min\left(10,\ S_{T,i} + S_{E,i}\right) \\
\text{Final Score} &= \left\lceil \frac{1}{3} \sum_i \text{Score}_i \right\rceil
\end{aligned}
$$

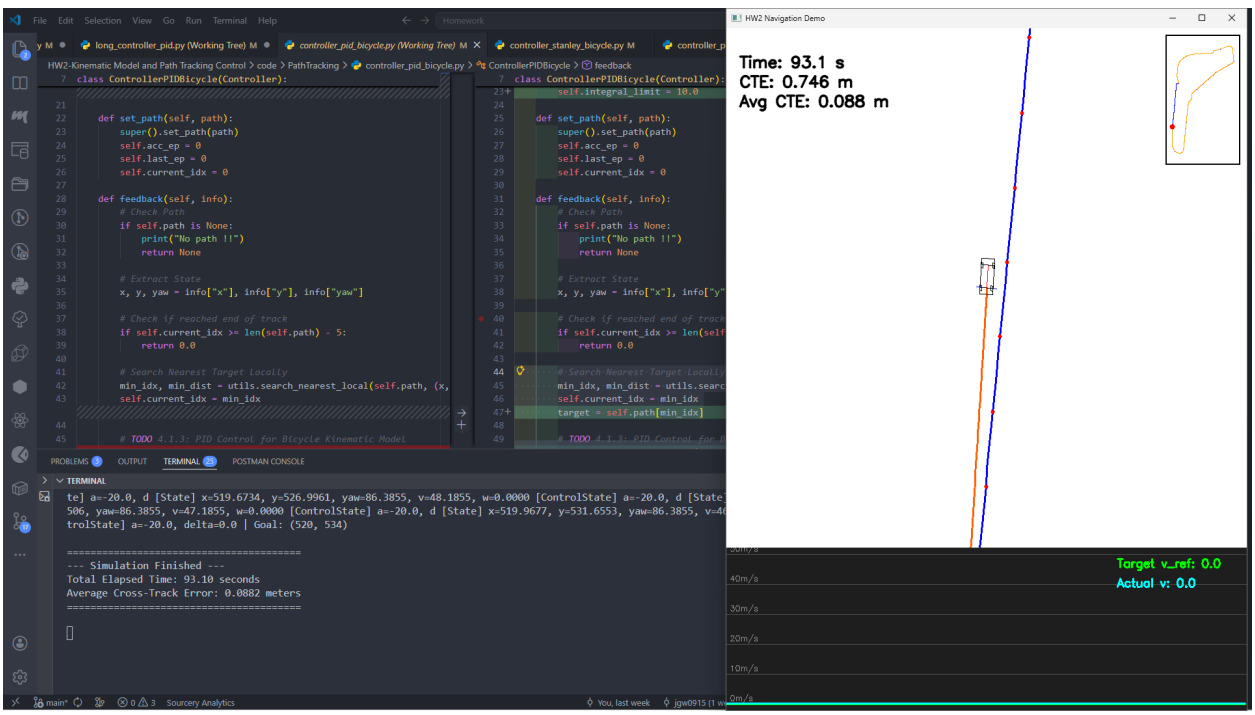
Here, $k_T = 0.5$ and $k_E = 2.0$ are penalty coefficients. Note that tracking error is penalized more heavily than completion time, so prioritize accuracy over speed. Try your best!

✎ Write down your command for this challenge, and put the screenshot of your result.

```
$ python navigation.py -s bicycle -c stanley -t [Silverstone|Suzuka|Monza]
```







#NewFeature Modifications

The following code segments were modified outside the original TODO blocks, so they were marked with `#NewFeature`. Their purposes are summarized below.

- **controller_pid_basic.py**

Added preview-related tuning parameters and speed-aware state usage for the PID controller. This change allows the controller to adjust its lookahead behavior based on vehicle speed, improving stability and path tracking on curved F1 tracks.

- **controller_pid_bicycle.py**
  Added steering output limits and integral limits. These constraints prevent overly aggressive steering commands and reduce integral windup, which improves the robustness of the bicycle PID controller.
- **controller_pure_pursuit_basic.py**
  Added a speed-dependent lower-bounded lookahead distance. This helps the pure pursuit controller remain stable at both low speed and high speed.
- **controller_pure_pursuit_bicycle.py**
  Added steering saturation and a speed-aware minimum lookahead distance. These changes keep the steering command within the physical limit and improve path tracking consistency across different track sections.
- **controller_stanley_bicycle.py**
  Added a steering saturation limit. This ensures that the Stanley controller output does not exceed the bicycle model steering constraint.
- **controller_lqr_bicycle.py**
  Replaced the nearest-point search with a local nearest-target search and normalized the copied target yaw. This improves forward progress along the path and avoids unstable behavior caused by targeting inappropriate points on curved tracks.
- **long_controller_pid.py**
  Added integral clamping to the longitudinal PID controller. This reduces integral windup during acceleration and deceleration, leading to smoother speed tracking.
- **benchmark_f1_challenge.py**
  Added a headless benchmark utility for automatically evaluating different model and controller combinations on Silverstone, Suzuka, and Monza. This script was used to compare performance efficiently.
- **run_f1_challenge_all.bat**
  Added a convenience batch script for launching the benchmark utility. This simplifies repeated testing of all challenge configurations.

2. ✎ (Extra up to +5 pts) Describe what you have tried to improve the performance:
   To improve the performance of the system in the F1 Track Challenge, I mainly focused on refining the implementation of the basic PID controller for both the basic model and the differential drive model, with the primary goal of reducing the average cross-track error (avg_cte). Since the grading policy penalizes tracking error more heavily than completion time, I prioritized path-following accuracy over aggressive speed. Specifically, I adjusted the PID control logic to produce smoother and more stable steering corrections, reducing oscillation when the vehicle approached the reference path and preventing overreaction on sharp curves. I also improved the balance among the proportional, integral, and derivative terms so that the controller could respond quickly to lateral deviation while avoiding excessive correction and accumulated error. For the differential drive model, I further adapted the PID-based steering response to better match its motion characteristics, allowing the vehicle to track the racing line more consistently through turns. For the basic model, I emphasized stable error feedback and gradual heading correction so that the controller could maintain a

smaller deviation over the full lap. Overall, these implementation-level improvements made the controller more robust across Silverstone, Suzuka, and Monza, and successfully reduced avg_cte, which in turn improved the final challenge performance.

6 Feedback (Optional)

1. (Extra up to +5 pts) We would love to hear your feedback on this homework! Please write down your thoughts, any difficulties you encountered, or suggestions for improving this assignment. Your feedback will be highly valuable for future improvements.
   This homework provided a valuable opportunity to learn and compare different controller logics and their corresponding mathematical formulations; however, I occasionally encountered difficulties when translating these concepts into code, mainly due to misunderstandings of variable units (e.g., angle, velocity, and time), which sometimes led to unexpected behavior in the controller. In addition, I gained practical experience in tuning parameters such as the gains in PID control and the state and control weighting matrices in LQR, which helped me better understand their impact on system performance. Nevertheless, achieving strong performance in the F1 challenge was still challenging, as it required repeated trial-and-error adjustments and continuous modifications to the controller implementation. Overall, I found this assignment both challenging and rewarding, but I would suggest providing clearer guidance or examples regarding unit consistency and parameter tuning strategies to help students implement the controllers more effectively.

Assignment Checklist

Please check the boxes (□ → ■) which you have completed

Done	No.	Type	Task	Points
■	2.1.1	✎	Answer questions (yaw unit, update) about `kinematic_basic.py`	4
■	2.1.2	✎	List constraints about `simulator_basic.py`	2
■	2.2.1	✎	List constraints regarding `simulator_different_drive.py`	2
■	2.2.2	🐍	Complete # TODO 2.2.2 in `navigation.py` for differential drive	4
■	2.3.1	🐍	Complete `step()` in `KinematicModelBicycle`	5
■	2.3.2	✎	List constraints regarding `simulator_bicycle.py`	3
■	3.1	🐍	Complete velocity profiling in `trajectory_generator.py` (Speed limit 4 pts + Smoothing 8 pts)	12
■	3.2	🐍	Implement Longitudinal PID control in `navigation.py`	8
■	4.1.1	✎	Identify error term in `PathTracking/controller_pid_basic.py`	2
■	4.1.2	🐍/✎	Tune PID gains in `controller_pid_basic.py` and describe tuning process	5
■	4.1.3	🐍/✎	Completed `PathTracking/controller_pid_bicycle.py`	5
■	4.2.1	🐍	Complete `PathTracking/controller_pure_pursuit_bicycle.py`	8
■	4.3.1	🐍	Complete `PathTracking/controller_stanley_bicycle.py`	8
■	4.4.1	🐍	Complete `PathTracking/controller_lqr_bicycle.py`	8
■	4.4.2	✎	Write down how you choose the weight matrices Q and R	4
■	4.4.3	✎	Complete linear system of Equation (19)	4
■	4.4.4	🐍	Complete `PathTracking/controller_lqr_bicycle.py`	6
■	5.1	🐍/✎	F1 Track Challenge (Choose best model & controller)	10
■	5.2	✎	Describe attempts to improve performance (Extra)	+5
■	6.1	✎	(Optional) Provide feedback and suggestions	+5
			Total Score	**100** (+10)